Introduction to C++

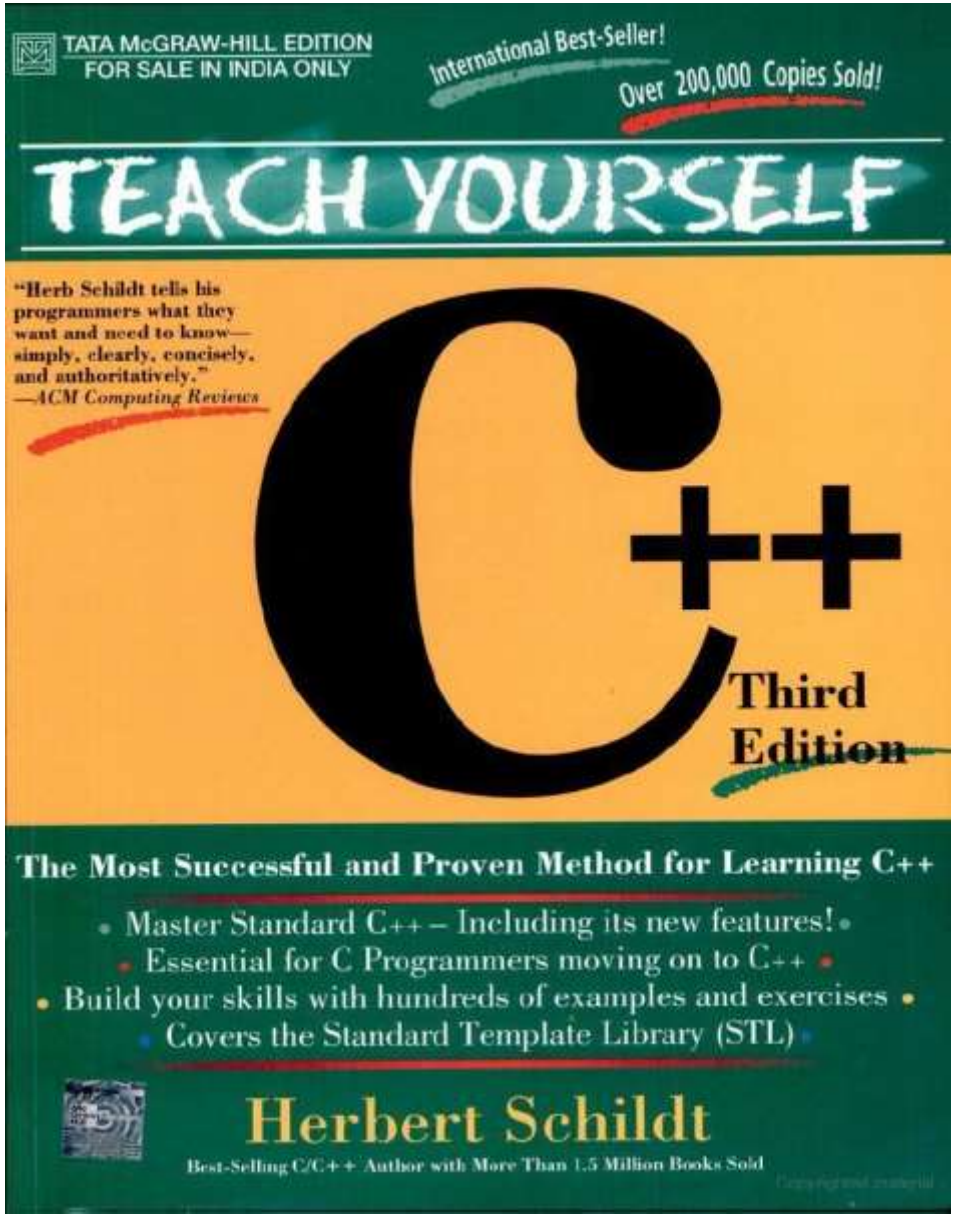
Contents

- C++
- Our Reference Book
- Object Oriented Programming
- Structure of simple C++ programs
- History of C++
- Classes: A First Look
- Introducing Function Overloading
- Differences between C and C++
- Exercises

C++

- A programming language is a formalized ***set of rules and instructions*** that allow humans to communicate with computers.
- Each programming language has its own ***syntax*** (grammar and rules), semantics (meaning of code), and features that make it suitable for certain types of tasks.
- A program refers to a ***set of instructions*** written in a programming language that ***tells a computer*** how to perform a specific task or achieve a particular goal.
- C++ is a versatile and widely used programming language that extends the capabilities of the C programming language.
- ***IDE*** stands for Integrated Development Environment. It is a software application to assist programmers and developers in writing, testing, debugging, and deploying software applications. IDEs usually come with built-in ***compilers*** that translate high-level programming code written by developers into machine-readable code.

Our Reference Book



Object Oriented Programming

- OOP stands for Object-Oriented Programming. It's a programming paradigm that organizes code and data in a way that ***models real-world entities*** and their interactions. OOP aims to make software development more modular, maintainable, and easier to understand.
- Here are some key concepts in Object-Oriented Programming:
  - **Objects:** Objects are ***instances*** of classes. They represent real-world entities and encapsulate data (attributes) and behavior (methods/functions) that operate on that data.
  - **Classes:** Classes are blueprints or ***templates for creating objects***. They define the structure and behavior of objects. A class specifies the attributes an object will have and the methods it can perform.

Object Oriented Programming

- Here are some key concepts in Object-Oriented Programming:
  - **Encapsulation:** Encapsulation is the practice of ***bindling data (attributes) and methods that operate on that data within a single unit (an object).*** It restricts direct access to the internal state of an object and enforces interaction through well-defined methods, promoting data integrity and reducing unintended interference.
  - **Inheritance:** Inheritance allows one class (subclass or derived class) to ***inherit properties and behaviors*** from another class (superclass or base class). This enables code reuse and the creation of hierarchies of classes with increasing specialization.
  - **Polymorphism:** Polymorphism allows different objects to respond to the same method call in a way that's appropriate for their specific types. This promotes code flexibility and extensibility.
  - **Abstraction:** Abstraction involves ***simplifying complex reality*** by modeling classes based on their essential properties. It hides the unnecessary details and focuses on the relevant aspects.

Structure of simple C++ programs

```
//simple program in C++

#include<iostream>

using namespace std;

int main()

{

        cout << “Welcome To IIUC”;

        return 0;

}
```

Output : Welcome to IIUC

Object Oriented Programming

- **Namespace:** When you include a new-style header in your program, the contents of that header are contained in the ***std*** namespace. A namespace is simply a ***declarative region***. The purpose of a namespace is to localize the names of identifiers to ***avoid name collisions***.
- **C++ Console I/O:**
  - In C++, I/O is performed using I/O operators.
  - The Output operator is << and the input operator is >>.
  - ***cout*** is a predefined output stream object that is part of the Standard Input/Output Library (iostream). It's used to send output to the standard output (usually the console or terminal) in C++ programs.
  - ***cin*** is a predefined input stream object that is also part of the Standard Input/Output Library (iostream). It's used to read input from the standard input (usually the keyboard or other input source) in C++ programs.

History of C++

- C++, developed by Bjarne Stroustrup at Bell Labs in the late 1970s, is an extension of C designed to support object-oriented programming while maintaining C compatibility.
- The first C++ compiler, "Cfront," emerged in 1983. The language gained its name in 1985.
- C++ combines C's low-level features with OOP. It's applied in diverse domains, such as systems programming and game development.
- Bjarne Stroustrup's book "The C++ Programming Language" played a key role in its expansion.

Classes: A First Look

- The single most important feature of C++ is the class.
- The class is the mechanism that is used to create objects.
- A class is declared using the **class** keyword.
- Functions and variables declared inside a declaration are said to be *members* of that class.
- To declare public class members, the *public* keyword is used, followed by a colon.

Introducing Function Overloading

- After classes, perhaps the next most important and pervasive C++ feature is ***function overloading*** by which C++ achieves one type of *polymorphism*.
- In C++, two or more functions can share the same name as long as either the type of arguments differs or the number of their arguments differs or both.
- When two or more functions share the same name, they are said to be *overloaded*.
- Overloaded functions reduce the complexity of a program by allowing related operations to be referred to by the same name.

Differences between C and C++

- C and C++ are both programming languages with some similarities, as C++ was developed as an extension of the C language. However, there are several key differences between the two languages. Here are some of the main differences:
- Object-Oriented Programming:
  - C does not have built-in support for object-oriented programming. It lacks classes, objects, inheritance, and other OOP concepts.
  - C++ is designed to support object-oriented programming. It introduces classes, objects, inheritance, polymorphism, and encapsulation.
- Standard Libraries:
  - C has a standard library (C Standard Library) that provides functions for common tasks like string manipulation and memory management.
  - C++ extends the C Standard Library with the C++ Standard Library, which includes the C Standard Library functions plus additional features tailored to C++ like the Standard Template Library (STL) for data structures and algorithms.

Differences between C and C++

- Memory Management:
  - Memory management in C relies on manual allocation and deallocation using functions like malloc and free.
  - C++ provides automatic memory management through constructors and destructors. It introduces ***new*** and ***delete*** operators for dynamic memory allocation and deallocation.
- Function Overloading:
  - C does not support function overloading. You cannot define multiple functions with the same name but different parameter lists.
  - C++ supports function overloading, allowing you to define multiple functions with the same name but different parameter lists.
- Operator Overloading:
  - C does not support operator overloading. Operators have fixed meanings.
  - C++ supports operator overloading, allowing you to redefine the behavior of operators for user-defined types.

Differences between C and C++

- Programming Paradigm:
  - C is a procedural programming language. It focuses on functions and structured programming.
  - C++ is a multi-paradigm programming language that supports procedural, object-oriented, and generic programming.
- Namepaces:
  - C does not have namespaces.
  - C++ introduces namespaces, which help avoid naming conflicts and organize code into separate logical units.
- Compatibility:
  - C code is usually more portable between different platforms and compilers due to its simpler nature.
  - C++ can sometimes be less portable due to its additional features and complexity.

The choice between C and C++ depends on factors such as project requirements, programming paradigms, and desired levels of control and complexity.

Exercises

- Overload the display function in the Person class so that it can display either the name or the name and age of the person, based on the number of arguments passed.